

Guion docente sobre clases y objetos en C++

Guion Docente: Clases y Objetos en C++

1. Introducción

Una **clase** es un modelo que describe atributos y métodos de un tipo de objeto. Un **objeto** es una instancia concreta creada a partir de una clase.

2. Ejemplo de abstracción: Clase Auto

A continuación se muestra cómo identificar atributos y métodos mediante el proceso de abstracción.

Atributos identificados

- Marca
- Modelo
- Año
- Color
- Velocidad

Métodos identificados

- acelerar()
- frenar()

Código comentado

```
class Auto {  
public:  
    std::string marca; // Atributo: marca del auto  
    std::string modelo; // Atributo: modelo del auto  
    int año; // Atributo: año de fabricación  
    std::string color; // Atributo: color del auto  
    int velocidad; // Atributo: velocidad actual  
  
    void acelerar() { // Método: incrementa la velocidad  
        velocidad += 10;  
    }  
  
    void frenar() { // Método: reduce la velocidad  
        velocidad -= 10;  
    }  
};
```

3. Encapsulamiento y niveles de acceso

El encapsulamiento permite proteger los atributos internos de la clase.

Niveles de acceso

- **public:** accesible desde cualquier parte del programa.
- **private:** accesible solo dentro de la clase.
- **protected:** accesible dentro de la clase y clases derivadas.

Código comentado

```
class Auto {  
private:  
    int velocidad; // Atributo privado: velocidad protegida  
  
public:  
    Auto() : velocidad(0) {} // Constructor: inicializa velocidad en 0  
  
    void acelerar() { // Método: incrementa la velocidad  
        velocidad += 10;  
    }  
  
    void frenar() { // Método: reduce la velocidad  
        velocidad -= 10;  
    }  
  
    int getVelocidad() const { // Método: devuelve la velocidad actual  
        return velocidad;  
    }  
};
```

4. Ejemplo de abstracción: Clase Persona

Atributos identificados

- nombre
- edad

Métodos identificados

- presentarse()
- cumplirAnios()

Código comentado

```
class Persona {
private:
    std::string nombre; // Atributo privado: nombre de la persona
    int edad; // Atributo privado: edad de la persona

public:
    Persona(const std::string& nombre, int edad) // Constructor
        : nombre(nombre), edad(edad) {} // Inicializa atributos

    void presentarse() const { // Método: muestra presentación
        std::cout << "Hola, soy " << nombre << " y tengo " << edad << " años.";
    }

    void cumplirAnios() { // Método: incrementa la edad
        edad++;
    }
};
```

5. Instanciación de objetos

Crear un objeto significa generar una instancia de una clase.

Código comentado

```
Persona p("Paulo", 35); // Instancia de Persona con nombre y edad
p.presentarse(); // Llama al método presentarse
p.cumplirAnios(); // Incrementa la edad
p.presentarse(); // Muestra la nueva edad
```

6. Comparación de estilos de diseño

Clase simple sin encapsulamiento

```
class Punto {
public:
    int x; // Coordenada X
    int y; // Coordenada Y
};
```

Clase con encapsulamiento

```
class Punto {  
private:  
    int x; // Coordenada X privada  
    int y; // Coordenada Y privada  
  
public:  
    Punto(int x, int y) : x(x), y(y) {} // Constructor  
  
    void mover(int dx, int dy) { // Método: mueve el punto  
        x += dx;  
        y += dy;  
    }  
  
    void mostrar() const { // Método: muestra coordenadas  
        std::cout << "(" << x << ", " << y << ")";  
    }  
};
```

7. Conclusión

Este guion permite explicar de manera gradual:

- Qué es una clase y un objeto.
 - Cómo identificar atributos y métodos.
 - Cómo definir niveles de acceso.
 - Cómo instanciar objetos.
 - Cómo comparar diferentes estilos de diseño.
-